STP 2026 · TEAM 06 · EXPLAINER

How *STP_Spacearm* works — explained with simple maths

A guide to the teammate's code: what each part does, the equations behind it in plain words, and how it compares with the Neural IK project.

Source: repository `akashrayganguly/STP_Spacearm` (commit a15a269) and its deck `Spacearm_deck_v2.pptx`. Read only, nothing was re-run: all numbers are the teammate's own reported results; figures are taken from the repository's `reports/figures/`. Neural IK numbers are the ones re-measured for the Neural IK deliverables.

1 The idea in one page

A small satellite floats in space with a 7-joint robot arm on its top deck. When the arm moves, the satellite turns slightly the other way (like a skater pushing off). The task has two levels:

Level 1 · Plan
Given the arm's current joint angles and a target point, produce a smooth joint-space path that never touches the satellite or the arm itself.

→

Level 2 · Fly
Move the arm there for real, every 0.1 s, while a ball suddenly appears, sensors are slightly wrong and motors are weak — and the satellite keeps turning.

The code is built from a few small pieces that each solve one sub-problem:

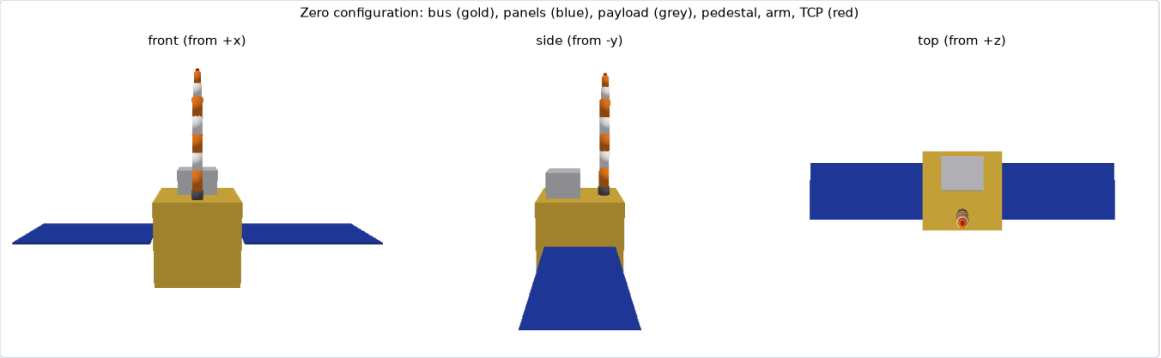
Piece	Plain words	File
World model	Floating satellite + arm in PyBullet, zero gravity	`robot_model.py`, `sim.py`
Exact arm maths	Joint angles → tool-tip position (forward kinematics)	`kinematics.py`
Dataset	200,000 poses, each with its exact distance to contact	`datagen.py`
DistanceNet	Neural "how far am I from touching, and which way is safer?"	`models/distance_net.py`
TrajNet	Neural "suggest a whole smooth path in one shot"	`models/traj_net.py`, `losses.py`
Planner	Guess → improve → polish → verify	`planner.py`
Reflex	Hand-written controller: aim, lean off, push off	`control.py`
RL correction	Tiny network that learns what the reflex gets wrong	`rl/ppo_lag.py`, `envs/space_reach_env.py`
Safety shield	Last check before the motors: is the next 0.2 s safe?	`safety/shield.py`

Symbols used in this document

Symbol	Meaning	Symbol	Meaning
$q \in \mathbb{R}^7$	the 7 joint angles (radians)	$\dot{q}$	joint speeds (rad/s)
x_{tip}	tool-tip position (3 numbers, metres)	x^*	target point
$d_{\text{body}}, d_{\text{self}}$	gap arm↔satellite, arm↔arm (negative = overlapping)	$c(q)$	combined clearance (the smaller gap, smoothed)
∇c	gradient: which way to turn the joints to gain clearance	J	Jacobian: how the tip moves when the joints move
$P_0, \dots, P_7$	Bézier control poses of a path	$s \in [0,1]$	position along the path (0 = start, 1 = goal)
$a \in [-1,1]^7$	the RL policy's output (correction)	π_ϕ	the policy network with weights ϕ
$r_t,\ c_t$	reward and cost at step t	λ	safety price (Lagrange multiplier)
$\sigma(z)$	sigmoid $1/(1+e^{-z})$	$\|v\|$	length of a vector

2 The world: a floating satellite

The robot is generated from one settings file (`configs/default.yaml`): a 1 m cube bus (150 kg), two solar panels, a payload box and a 7-link arm made of capsules (18 kg). Total 212 kg; the arm is 8.5 % of the mass. Joint axes alternate z-y-z-y-z-y-z (shoulder 3, elbow 1, wrist 3), like a KUKA iiwa.

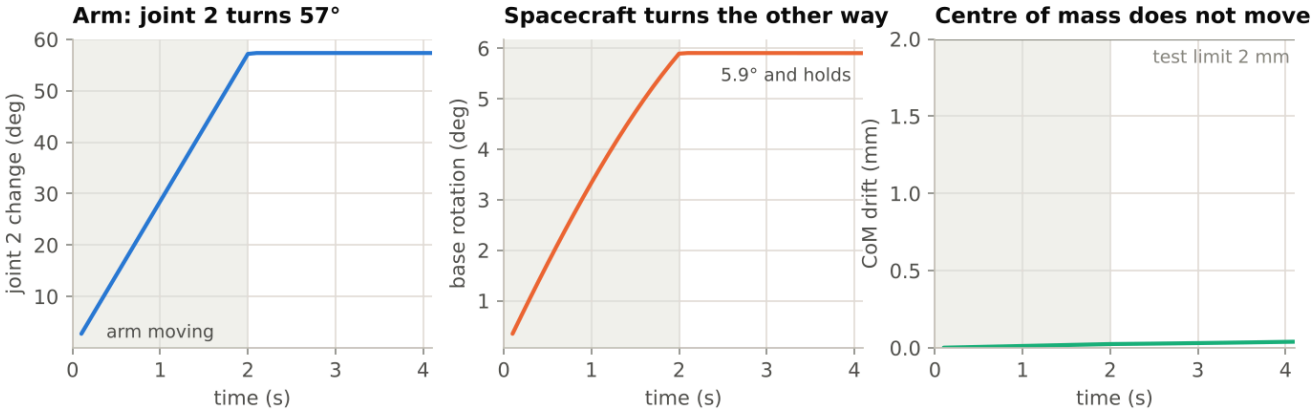


The simulated satellite: bus (gold), solar panels (blue), payload (grey), pedestal and arm. Figure from the teammate's repository.

PyBullet runs with **zero gravity and no damping**, so nothing slows motion down and momentum is never lost. Starting at rest, the total momentum of the whole system stays zero:

$$\underbrace{\sum_i m_i\, v_i}_{\text{linear momentum}} = 0 \qquad \underbrace{\sum_i \left(I_i\, \omega_i + m_i\, r_i \times v_i\right)}_{\text{angular momentum}} = 0$$

In words: **if the arm swings one way, the satellite must turn the other way** so that the totals stay zero. The centre of mass does not move at all. In their test the arm turns joint 2 by 57° and the satellite turns 5.9° the other way.



Arm moves (left) → satellite turns the other way (middle) → centre of mass stays put (right). Figure from the teammate's repository.

> **WHY IT MATTERS**
> A plan computed in the satellite's own frame is perfect in that frame, but when it is played on the floating satellite the tip misses a target fixed in space by **17.2 cm** (median, max 33.3 cm). This single number is why Level 2 exists.

Timing: physics runs at 240 Hz, decisions at 10 Hz, so one decision lasts 24 physics steps (0.1 s). Joints are velocity-controlled, at most 0.5 rad/s.

3 Arm maths: forward kinematics and the Jacobian

Forward kinematics (FK) answers: *given the joint angles, where is the tool tip?* It walks along the arm, one joint at a time. Start at the pedestal with position t and orientation R (a 3×3 rotation matrix), then for each joint i:

$$t \leftarrow t + R\,e_z\,\ell_i, \qquad R \leftarrow R \cdot \mathrm{Rot}(\mathrm{axis}_i,\, q_i), \qquad x_{\mathrm{tip}} = t_7 + R_7\,e_z\,\ell_{\mathrm{tool}}$$ FK chain

Here ℓ_i are the link lengths (24, 20, 20, 20, 20, 14, 12 cm), $\ell_{\mathrm{tool}} = 15$ cm and $e_z = (0, 0, 1)$: "move along the link, then turn by the joint angle". It is written in PyTorch, so it is **differentiable**: the networks can learn *through* it. It matches PyBullet to 0.00012 mm.

The **Jacobian** J (3×7) says how the tip moves for small joint motions:

$$\Delta x_{\mathrm{tip}} \;\approx\; J(q)\,\Delta q \qquad (3 \text{ numbers out}, 7 \text{ numbers in})$$

Because 7 joints control only 3 numbers (position), there are many ways to reach the same point. The "spare" freedom is called the **null space**; it is used later to lean the elbow away from the satellite without moving the hand.

4 Making the data

No dataset exists for this robot, so it is generated. For each random pose q, PyBullet measures exact, noise-free labels:

$$q \longmapsto \big(\, d_{\text{body}}(q),\ d_{\text{self}}(q),\ x_{\text{tip}}(q) \,\big), \qquad d < 0 \ \Rightarrow\ \text{overlapping (collision)}$$

- **150,000 uniform poses** (75 %): every joint uniform inside the central 98 % of its range.
- **50,000 near-contact poses** (25 %), made by "nudging": take a pose that is already within 5 cm of contact, add small random noise, keep it only if it is still within 5 cm:

$$q' = q + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0,\ 0.05^2)\ \text{rad per joint}, \qquad \text{keep if } \big|\min(d_{\text{body}}, d_{\text{self}})\big| < 5\ \text{cm}$$

- **Test set**: 20,000 uniform poses with a different random seed, never used for training.

Effect: the share of training poses near a surface goes from 7.7 % to 30.9 %, so the network sees many examples exactly where decisions are made. Generation takes 38 s (0.19 ms per pose) and is byte-for-byte reproducible from the seed.

5 DistanceNet: "how close am I to touching?"

PyBullet can measure the gap, but only as a number (0.16 ms per pose) — it does not say *which way* to move to be safer. A small smooth network learns the gap and, through its gradient, the direction.

Network

$$\underbrace{[\sin q,\ \cos q]}_{14\ \text{inputs}} \xrightarrow{256} \text{SiLU} \xrightarrow{256} \text{SiLU} \xrightarrow{256} \text{SiLU} \longrightarrow (\hat{d}_{\text{body}},\ \hat{d}_{\text{self}}), \qquad \text{SiLU}(z) = z\,\sigma(z)$$

Sine and cosine are used because angles wrap around (+180° and −180° are the same). SiLU is a smooth activation, so the gradient ∇c is smooth too. 135,938 parameters (0.5 MB).

Clearance: a cautious soft minimum

$$c(q) \ =\ -\tau\,\log\!\Big(e^{-\hat{d}_{\text{body}}/\tau} + e^{-\hat{d}_{\text{self}}/\tau}\Big), \qquad \min(\hat{d}_{\text{body}}, \hat{d}_{\text{self}}) - \tau\ln 2 \ \le\ c \ \le\ \min(\hat{d}_{\text{body}}, \hat{d}_{\text{self}})$$

soft-min, τ = 1 cm

> **WORKED EXAMPLE**
> If $\hat{d}_{\text{body}} = 2$ cm and $\hat{d}_{\text{self}} = 10$ cm: $c = -\ln(e^{-2} + e^{-10}) \approx 1.9997$ cm — almost exactly the smaller gap.
> If both are 3 cm: $c = 3 - \ln 2 \approx 2.31$ cm — slightly *below* the true gap, i.e. on the safe side. Unlike a hard min, its gradient does not jump when the closest surface changes.

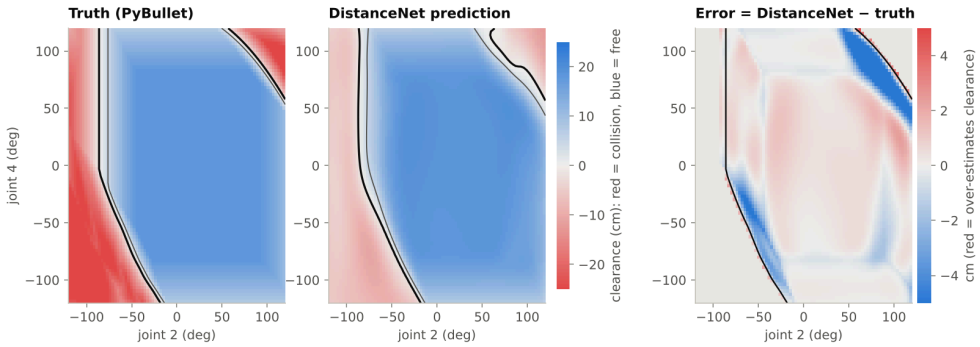
Training loss: precise where it matters

$$\mathcal{L} = \text{mean}\,\big[\, w \cdot \ell_\beta(\hat{d} - d)\,\big], \qquad \ell_\beta(e) = \begin{cases} \dfrac{e^2}{2\beta} & |e| < \beta \\ |e| - \dfrac{\beta}{2} & \text{otherwise} \end{cases}, \qquad w = \begin{cases} 10 & |d| < 5\ \text{cm} \\ 1 & \text{otherwise} \end{cases}$$

β = 2 cm

In words: small mistakes are squared (be precise), big mistakes count only linearly (a few deep collisions cannot dominate), and mistakes near a surface count 10×. Targets are clipped to −5…+30 cm: beyond that no decision changes. Training: AdamW, learning rate 2×10^{-3} with cosine decay, 60 epochs, batch 1,024, 49 s on 4 CPU threads.

Test (20,000 unseen poses)	Target	Result
Mean error, all poses	≤ 1.0 cm	**0.68 cm**
Mean error within 10 cm of contact	≤ 2.0 cm	**1.22 cm**
Collide / free called correctly	≥ 97 %	**98.3 %**
False "safe" (predicts > 3 cm while colliding)	≤ 0.5 %	**0.12 %**



A 2-D slice through joint space: PyBullet truth (left), DistanceNet (middle), error (right). Black line = contact. Figure from the teammate's repository.

6 TrajNet: one forward pass proposes a whole path

Input: the start pose and the target point. Output: 35 numbers that describe a smooth path.

$$\underbrace{[\sin q_0,\ \cos q_0,\ x^*]}_{17} \longrightarrow \text{MLP}\ (512 \times 3,\ \text{SiLU}) \longrightarrow \underbrace{u}_{35 = 7\ (\text{goal}) + 4\times 7\ (\text{middle})}$$

Step 1 — squash into the joint limits

$$q = m + h\,\tanh(u), \qquad m = \frac{q_{\max} + q_{\min}}{2}, \quad h = \frac{q_{\max} - q_{\min}}{2}$$

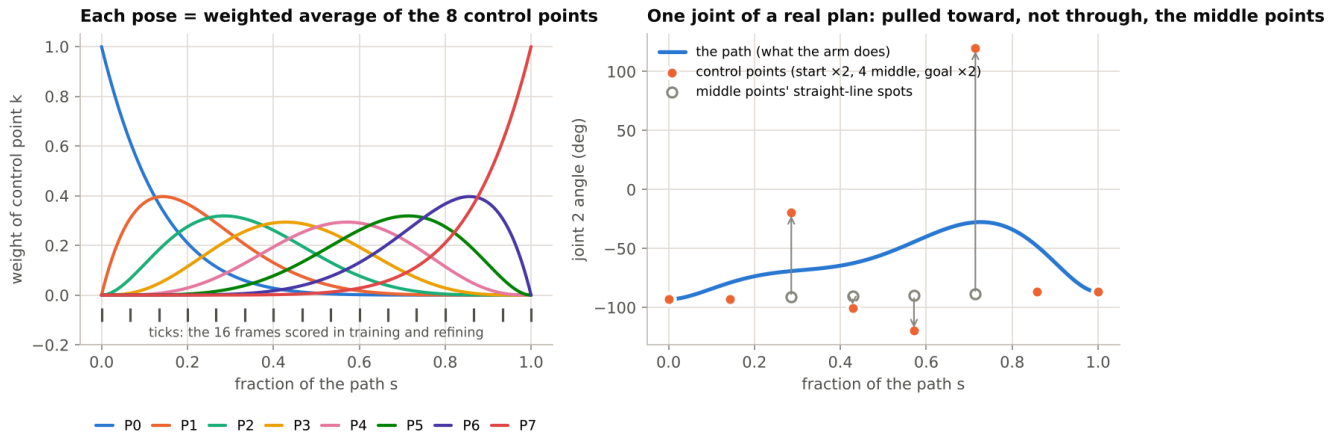
Because $\tanh$ is always between −1 and 1, every control pose lies inside the joint limits — whatever the network outputs.

Step 2 — a Bézier curve through 8 control poses

$$q(s) = \sum_{k=0}^{7} \binom{7}{k} s^k (1-s)^{7-k}\, P_k, \qquad P_0 = P_1 = q_0, \quad P_6 = P_7 = q_{\text{goal}}, \quad P_2..P_5 = \text{middle poses}$$

degree-7 Bézier

- Each pose on the path is a **weighted average** of the 8 control poses (the weights add up to 1). So if all control poses are inside the limits, the whole path is too ("convex hull" property).
- The speed at the ends is $q'(0) = 7(P_1 - P_0) = 0$ and $q'(1) = 7(P_7 - P_6) = 0$: the arm **starts and stops at rest**.



Left: the 8 weights along the path. Right: one joint of a real plan, pulled toward (not through) the middle points. Figure from the teammate's repository.

Step 3 — learning without example paths

A target point can be reached by infinitely many poses and paths; copying examples would average several right answers into a wrong one. So TrajNet is **self-supervised**: it lowers a score that checks the outcome.

$$J(\theta) = \underbrace{100\,\left\|\text{FK}(q_{\text{goal}}) - x^*\right\|^2}_{\text{reach the target}} + \underbrace{1000 \cdot \frac{1}{16}\sum_{k=1}^{16} \max\left(0,\ 0.05 - c(q(s_k))\right)^2}_{\text{stay 5 cm clear (DistanceNet)}} + \underbrace{0.1 \sum \|\Delta q\|^2}_{\text{smooth}}$$

score to minimise

WORKED EXAMPLE (UNITS: METRES)

A 1 cm reach miss costs $100 \times 0.01^2 = 0.01$. Being 3 cm too close on average costs $1000 \times 0.03^2 = 0.9$ — ninety times more. So the network learns: **first never come close, then reach precisely**.

552,483 parameters; 10,000 training steps × 256 queries; 3 min 11 s on 4 CPU threads. Alone it succeeds on 85 % of random and 66 % of hard queries.

7 The planner: guess → refine → polish → verify

1 · Guess TrajNet, one pass, ≈ 3 ms → **2 · Refine ×30** Adam on the 35 numbers, ≈ 90 ms → **3 · Polish ×10** exact IK on the goal pose, ≈ 5 ms → **4 · Verify** PyBullet, 100 poses, ≈ 15 ms

Refine: improve this one path (test-time optimisation)

The network is frozen; the 35 numbers θ of *this* path are adjusted directly on the same score J:

$$\theta \leftarrow \theta - \mathrm{Adam}\big(\nabla_\theta J(\theta)\big), \qquad 30 \text{ steps } (100 \text{ on a retry})$$

Adam is gradient descent with a per-number step size that adapts to how noisy each gradient is. Refining buys clearance, but it can pull the tip 1–2 cm off the target.

Polish: put the tip exactly on the target

Only the goal pose moves, with damped least squares on the exact FK (10 iterations, $\lambda = 0.01$):

$$\Delta q = J^{\mathsf{T}}\big(JJ^{\mathsf{T}} + \lambda^2 I\big)^{-1}\big(x^{*} - x_{\mathrm{tip}}\big) \qquad \text{DLS IK step}$$

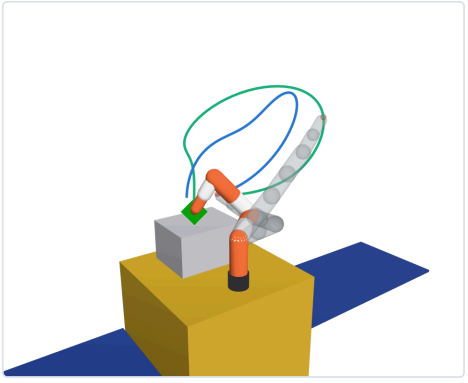
In words: "move the joints the smallest amount that moves the tip toward the target"; the $\lambda^2 I$ term keeps steps sane near awkward (singular) poses.

Verify

PyBullet checks 100 poses along the path: every clearance > 0, tip within 2 cm, all joints inside limits. If anything fails, plan once more with 100 refine steps (needed on 0.5 % of random queries).

Method (same queries)	Random 200: success / collide	Hard 100: success / collide	Tip error	Time
Baseline: PyBullet IK + straight line	92 % / 3.5 %	0 % / 100 %	0.14 cm	1 ms
TrajNet alone	85 % / 1.5 %	66 % / 6 %	1.02 cm	2–5 ms
+ refine	56 % / 0.5 %	54 % / 1 %	1.85 cm	≈ 90 ms
+ polish (no refine)	98 % / 1.5 %	93 % / 7 %	< 0.01 cm	8 ms
+ refine + polish	99.5 % / 0.5 %	100 % / 0 %	< 0.01 cm	≈ 99 ms
Full planner (+ verify, retry)	**100 % / 0 %**	**100 % / 0 %**	**< 0.01 cm**	**99 + 15 ms**

"Hard" = queries where the baseline collides (4.6 % of all queries). Refine buys clearance, polish buys precision; only together do they reach 100 %.



The planned path arcs over the payload (green and blue lines: tool-tip paths). Figure from the teammate's repository.

8 Level 2: flying the arm in a closed loop

One episode ("mission"):

- **Start:** random pose at least 5 cm from contact. **Target:** fixed in space, ≥ 20 cm away, within 85 % of reach.
- **Success:** $\|x_{\text{tip}} - x^*\| < 2$ cm and tip speed < 5 cm/s. **Failure:** any contact (body, self, ball) or 20 s.

Surprise	At full difficulty
Ball obstacle	80 % of episodes; 5–12 cm; appears 1–5 s in, in the way; half of them drift ≤ 3 cm/s
Encoder bias	each joint reads up to ±2° wrong
Weak motors	each joint gives 70–100 % of the commanded speed
Encoder slip	30 % of episodes: one joint reading jumps 3°
Sensor noise	encoders 0.1°, camera 5 mm, gyro 0.002 rad/s

The robot only sees **56 measured numbers**: joint readings and speeds, the camera vector tip→target, gyro spin, ball position and size, DistanceNet clearance and direction, and the previous and reflex commands.

The reflex: a hand-written baseline (no learning)

$$\dot{q}_{\text{reflex}} = \underbrace{J^{+} v}_{\text{aim}} + \underbrace{(I - J^{+}J)\, k_{\text{null}}\, w_s\, \nabla c}_{\text{lean off the satellite}} + \underbrace{k_{\text{obs}}\, w_o\, \nabla d_{\text{ball}}}_{\text{push off the ball}} \qquad \text{reflex prior}$$

$$v = \text{clip}_{\|\cdot\| \le 0.15\,\text{m/s}}\left(k_p\,(x^* - x_{\text{tip}})\right), \qquad J^{+} = J^{\mathsf{T}}\left(JJ^{\mathsf{T}} + 0.05^2 I\right)^{-1}$$

- **Aim:** move the tip straight toward the target ($k_p = 1$ /s, at most 15 cm/s).
- **Lean off:** $(I - J^{+}J)$ is the null-space projector — it keeps only joint motions that do *not* move the hand. The 7th joint swings elbow and wrist away from the satellite ($k_{\text{null}} = 0.5$ rad/s). The weight w_s ramps from 0 to 1 as clearance falls from 10 cm to 0.
- **Push off:** move the joints away from the ball ($k_{\text{obs}} = 0.5$ rad/s); w_o ramps 0 → 1 as the ball gap falls from 15 cm to 0.
- If any joint would exceed 0.5 rad/s, the whole vector is shrunk by the same factor.

> **WEAK SPOT**
> With a ball right on the straight line, "pull to the target" and "push off the ball" cancel out (a local minimum): the arm hovers 8–12 cm away until the 20 s run out.

9 The RL correction (residual PPO-Lagrangian)

Instead of learning everything from scratch (which gave 0 % success after 130k steps), a tiny network learns only a **bounded correction on top of the reflex**:

$$\dot{q}_{\text{cmd}} = \text{clip}\left(\dot{q}_{\text{reflex}} + 0.5 \cdot \text{fade} \cdot a\right), \qquad \text{fade} = \min\left(1,\ \frac{\|x_{\text{tip}} - x^*\|}{10\ \text{cm}}\right), \qquad a = \pi_\phi(o) \in [-1, 1]^7$$ command sent to motors

The correction has at most half the authority and fades out in the last 10 cm, so the reflex alone makes the precise final approach.

The policy network

$$o \in \mathbb{R}^{56} \ \longrightarrow\ 64\ (\tanh) \ \longrightarrow\ 64\ (\tanh) \ \longrightarrow\ a \in \mathbb{R}^7, \qquad 56 \cdot 64 + 64 + 64 \cdot 64 + 64 + 64 \cdot 7 + 7 + 7 = 8{,}270\ \text{parameters}$$

(The last 7 are the exploration noise levels.) It runs in 0.012 ms with NumPy.

Reward and cost

$$r_t = 10\,(D_{t-1} - D_t) \ - \ 0.01\,\|a_t - a_{t-1}\|^2 \ - \ 0.005 \ + \ 5 \cdot [\text{success}] \ - \ 0.02\,\|a_t\|^2$$ reward per step

D_t is the tip-to-target distance, so the first term pays for **progress**; the others discourage jerky corrections, wasted time and large corrections. Safety is not in the reward — it is a separate **cost**:

$$c_t = \begin{cases} 1 & \text{if anything is closer than 2 cm (a near miss)} \\ 0 & \text{otherwise} \end{cases}, \qquad \text{budget: } \mathbb{E}\Big[\sum_t c_t\Big] \le 1 \text{ per episode}$$

PPO: careful policy updates

$$L(\phi) = \mathbb{E}\Big[\min\left(\rho\,\hat{A},\ \text{clip}(\rho,\ 1-\varepsilon,\ 1+\varepsilon)\,\hat{A}\right)\Big], \qquad \rho = \frac{\pi_\phi(a \mid o)}{\pi_{\text{old}}(a \mid o)}$$ clipped objective, ε = 0.2

$\hat{A}$ (the **advantage**) says whether an action turned out better than expected. The clip stops any single update from changing the policy too much. Advantages are estimated with GAE ($\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$):

$$\hat{A}_t = \sum_{l \ge 0} (\gamma \lambda_{\text{GAE}})^l\, \delta_{t+l}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

The Lagrangian part: a self-adjusting safety price

$$\hat{A} = \frac{\hat{A}_r - \lambda\,\hat{A}_c}{1 + \lambda}, \qquad \lambda \ \leftarrow\ \text{clip}\left(\lambda + 0.05\,(\bar{C} - 1),\ 0,\ 10\right)$$ dual ascent

$\bar{C}$ is the average near-miss count per episode. If the policy goes over budget, the price λ rises and safety weighs more; if it stays under, λ falls. (In their runs λ never rose above 0.45, so most of the caution came from the reflex and the shield.)

Asymmetric critics: the teacher sees more than the student

Two value networks (V_r for reward, V_c for cost; 90 → 256 → 256 → 1, 89,345 parameters each) are used **only during training**. They see the 56 sensor numbers plus **34 hidden truths** (true joint angles, encoder bias, motor gains, satellite orientation, true clearances, ball velocity, time left). The policy that flies sees only the 56 sensor numbers. Training: 3.0 M steps, 4 simulators in parallel, difficulty ramped from 0 to 1 over the first half, ≈ 1 h 45 min on 4 vCPU.

10 The safety shield: one last check

Every command passes a model-based filter before it reaches the motors. It predicts the pose 0.2 s ahead (2 control steps) for three step sizes:

$$q'(s) = q + s \cdot \dot{q}_{\text{cmd}} \cdot 0.2\ \text{s}, \qquad s \in \{1,\ \tfrac{1}{2},\ \tfrac{1}{4}\}$$

$$h(q) = \min\Big(\, c(q) - 6\ \text{cm},\ \ g_{\text{ball}}(q) - 5\ \text{cm} \,\Big), \qquad \text{accept the largest } s \text{ with } h(q'(s)) \geq 0$$

safety function

If no step size is safe, the arm moves along ∇h (escape), or stops. The ball gap is pure geometry: each arm link is a capsule (a line segment with radius r_{link}) and the ball a sphere (centre b, radius r_b):

$$g_{\text{ball}} = \min_{\text{links}} \Big(\, \text{dist}\,(b,\ \text{segment}) - r_{\text{link}} - r_b \Big)$$

WHY 6 CM AND NOT 5 CM?

At 5 cm, 2 of 100 episodes grazed the satellite: after a joint slip the measured angles carried up to 3.6° of bias, so DistanceNet (fed with wrong angles) was ≈ 5.5 cm too optimistic. At 6 cm: 0 collisions in all scenarios, for about 2 points of success.

Cost of the shield: 0.18 ms per accepted command (3.3 ms when it has to escape). It intervenes on 9–15 % of steps.

11 Deployment

Only the policy flies. It is exported as three matrix multiplications with the observation normalisation built in, so no PyTorch is needed on board:

$$\hat{o} = \text{clip}\Big(\frac{o - \mu}{\sigma},\ -10,\ 10\Big), \qquad a = \tanh\Big(W_3\ \tanh\big(W_2\ \tanh(W_1 \hat{o} + b_1) + b_2\big) + b_3\Big)$$

Component (1 CPU thread)	p50	p99
Policy (NumPy / ONNX)	0.012 / 0.008 ms	0.029 / 0.024 ms
Reflex	0.30 ms	0.59 ms
Shield (accepted / escape)	0.18 / 3.3 ms	0.38 / 4.5 ms
Whole control step incl. physics	**4.5 ms**	**7.3 ms**

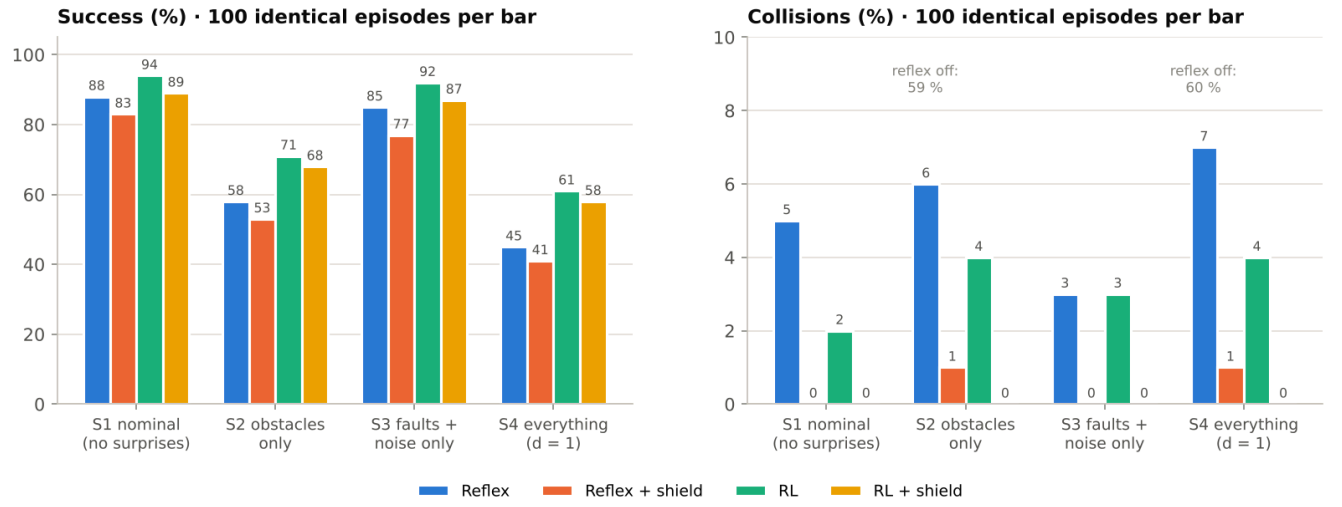
The control period is 100 ms, so the whole step uses under 8 % of the time budget.

12 Level 2 results

100 identical episodes per controller and scenario (seeds 10000–10099, never used for training):

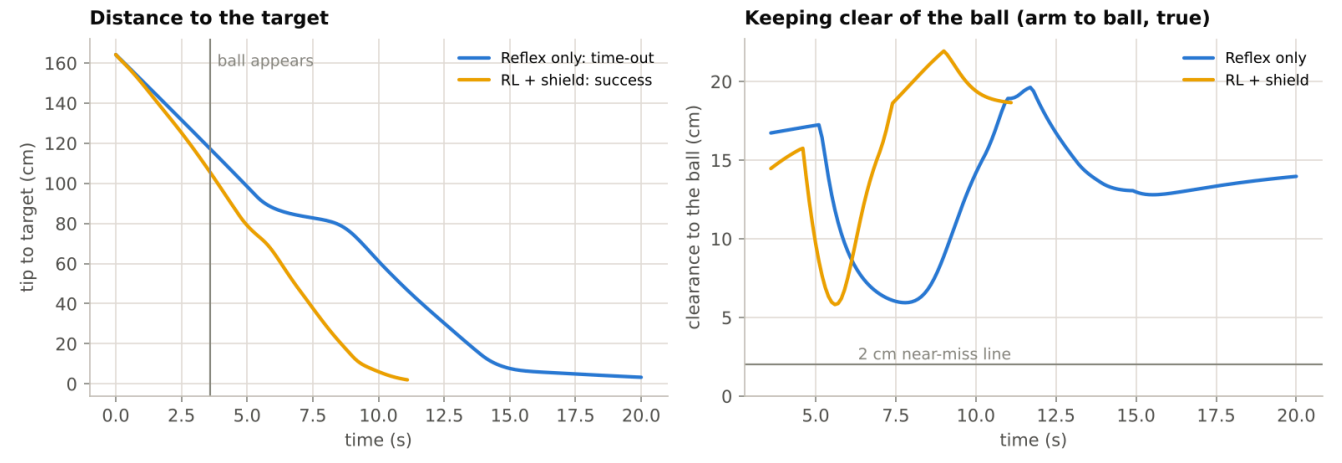
Scenario	Reflex	Reflex + shield	RL	RL + shield
S1 no surprises	88 % / 5 %	83 % / 0 %	94 % / 2 %	**89 % / 0 %**
S2 ball only	58 % / 6 %	53 % / 1 %	71 % / 4 %	**68 % / 0 %**
S3 faults + noise	85 % / 3 %	77 % / 0 %	92 % / 3 %	**87 % / 0 %**
S4 everything	**45 % / 7 %**	**41 % / 1 %**	**61 % / 4 %**	58 % / 0 %

Each cell: success / collisions. Time to reach in S4: 11.2 s (reflex) → 8.2 s (RL + shield).



Success (left) and collisions (right) per scenario and controller. Figure from the teammate's repository.

- The RL correction adds 6–16 points of success in every scenario and reaches targets ≈ 30 % faster.
- The shield removes every RL collision (13 → 0) for 3–5 points of success.
- Without the reflex's push-off, the reflex collides in ≈ 60 % of ball episodes: the classical layer matters.
- Hardest case: a static ball on the straight path (37 % reflex, 53 % RL + shield). The stretch goal of 70 % in S4 was not met.



One episode: the reflex hovers and times out; RL + shield goes around the ball and arrives. Figure from the teammate's repository.

13 Side by side with Neural IK

Both projects follow the same pattern — **a fast neural guess, an exact classical finish, an independent PyBullet check, and a learned correction on top of a classical controller** — but they solve different versions of the problem.

Neural IK in the same notation

$$[\,x^*,\ \text{quat}^*,\ q_{\text{seed}}\,] \in \mathbb{R}^{13} \longrightarrow (\sin\hat{q},\ \cos\hat{q}) \in \mathbb{R}^{12}, \qquad \hat{q}_i = \text{atan2}(\sin\hat{q}_i,\ \cos\hat{q}_i)\ \ (\pm 360^\circ \text{ into the limits})$$

network output → angle

$$\mathcal{L} = \text{MSE}\,(\sin/\cos) + 0.05 \cdot \text{mean}\ \sigma\big(C(\hat{q})\big)$$

collision-aware loss

C is a frozen yes/no self-collision classifier (6 → 64 → 64 → 1). Its probability is a soft penalty; it does not measure distance.

$$\Delta q = \big(J^{\mathsf{T}}J + \lambda I\big)^{-1} J^{\mathsf{T}} e, \qquad \lambda = \|e\|^2 + 10^{-6}, \qquad e = (\text{position error},\ \text{orientation error}) \in \mathbb{R}^6$$

Levenberg–Marquardt refinement

$$\Delta q_{\text{cmd}} = \Delta q_{\text{potential field}} + 2 \cdot \pi_\phi(o), \qquad o \in \mathbb{R}^{101},\ \ \pi_\phi : 2 \times 64\ \tanh,\ 11{,}078\ \text{parameters}$$

Level 2 command

	Neural IK (yours)	STP_Spacearm (teammate)
Robot / world	xArm6, 6 joints, fixed base, kinematic motion	7-joint arm on a 212 kg free-floating satellite, real physics
Level 1 target	full pose: position **and** orientation	point: position only
Level 1 output	one goal joint configuration	whole Bézier path + goal joints
Learning style	supervised (copy known joint angles)	self-supervised (score the outcome)
Collision knowledge	yes/no classifier, self only, 98.4 %	distance in cm + direction, body + self, 0.68 cm error
Classical finish	LM with restarts and collision-free retry	Adam refine + DLS polish
Check	final configuration in PyBullet	100 poses along the path in PyBullet
Level 1 result	100 % of 2,000 within **1 mm and 1°**, 0 % collisions, 12.9–37.8 ms	100 % of 300 within **2 cm**, 0 % collisions, ≈ 0.1 s
Level 2 algorithm	PPO residual on a potential field	PPO-Lagrangian residual on a reflex + safety shield
Level 2 hardest case	84 % reached / 2 % collided (200 episodes)	58 % reached / 0 % collided (100 episodes)

READ THE NUMBERS CAREFULLY

The Level 2 success rates are not directly comparable: the teammate's world is harder (floating base, noisy sensors, target fixed in space, 2 cm tip tolerance), while Neural IK's target tolerance is far stricter in Level 1 (1 mm and 1° on the full pose).

WHAT EACH COULD BORROW

From STP_Spacearm: a distance network with a gradient instead of a yes/no classifier; checking the whole path; a safety shield (removed all collisions there); a free-floating base and a 7th joint for the null-space lean-off; training critics on hidden simulator truths.
From Neural IK: full 6-D pose accuracy (orientation matters for grasping); tests from home and random starting poses; an "LM alone" ablation showing what the network really contributes.

14 All equations at a glance

Part	Equation
Momentum	$\sum m_i v_i = 0, \quad \sum (I_i \omega_i + m_i r_i \times v_i) = 0$
Forward kinematics	$t \leftarrow t + R e_z \ell_i, \;\; R \leftarrow R\,\mathrm{Rot}(\mathrm{axis}_i, q_i), \;\; x_{\mathrm{tip}} = t_7 + R_7 e_z \ell_{\mathrm{tool}}$
Jacobian	$\Delta x_{\mathrm{tip}} \approx J(q)\,\Delta q$
Near-contact data	$q' = q + \mathcal{N}(0, 0.05^2)$, keep if $\lvert \min d \rvert < 5$ cm
Clearance	$c = -\tau \log(e^{-d_b/\tau} + e^{-d_s/\tau}),\ \tau = 1$ cm
DistanceNet loss	$\mathrm{mean}[w\,\ell_\beta(\hat{d} - d)],\ \beta = 2$ cm, $w = 10$ near contact
Joint-limit squash	$q = m + h\tanh(u)$
Bézier path	$q(s) = \sum_{k=0}^{7} \binom{7}{k} s^k (1-s)^{7-k} P_k$
TrajNet score	$100 \lVert \mathrm{FK}(q_g) - x^* \rVert^2 + 1000\,\overline{\max(0, 0.05 - c)^2} + 0.1 \sum \lVert \Delta q \rVert^2$
Refine	$\theta \leftarrow \theta - \mathrm{Adam}(\nabla_\theta J)$
IK polish	$\Delta q = J^\mathsf{T}(JJ^\mathsf{T} + \lambda^2 I)^{-1}(x^* - x_{\mathrm{tip}})$
Reflex	$\dot{q} = J^+ v + (I - J^+ J) k_{\mathrm{null}} w_s \nabla c + k_{\mathrm{obs}} w_o \nabla d_{\mathrm{ball}}$
Residual command	$\dot{q}_{\mathrm{cmd}} = \mathrm{clip}(\dot{q}_{\mathrm{reflex}} + 0.5\,\mathrm{fade}\,a)$
Reward	$10(D_{t-1} - D_t) - 0.01 \lVert \Delta a \rVert^2 - 0.005 + 5[\mathrm{success}] - 0.02 \lVert a \rVert^2$
PPO	$\mathbb{E}[\min(\rho \hat{A}, \mathrm{clip}(\rho, 0.8, 1.2)\hat{A})]$
Lagrangian	$\hat{A} = (\hat{A}_r - \lambda \hat{A}_c)/(1 + \lambda), \;\; \lambda \leftarrow \mathrm{clip}(\lambda + 0.05(\bar{C} - 1), 0, 10)$
Shield	$h = \min(c - 6\text{ cm},\ g_{\mathrm{ball}} - 5\text{ cm}) \geq 0$ at $q + s\,\dot{q}_{\mathrm{cmd}}\,0.2$ s
Deployed policy	$a = \tanh(W_3 \tanh(W_2 \tanh(W_1 \hat{o} + b_1) + b_2) + b_3)$

Prepared for STP 2026 Team 06. All STP_Spacearm values are quoted from that repository's README, reports and configuration (`configs/default.yaml`); nothing in it was modified or re-run.